

ENGINEERING CHALLENGE

Full-Stack Engineering Assessment

A real-world engineering task built on an existing system.
Implement features across the full stack — progressively, stage by stage.

React 19 + TypeScript

React Query + Axios

.NET 8 + Dapper

SQLite

4 stages



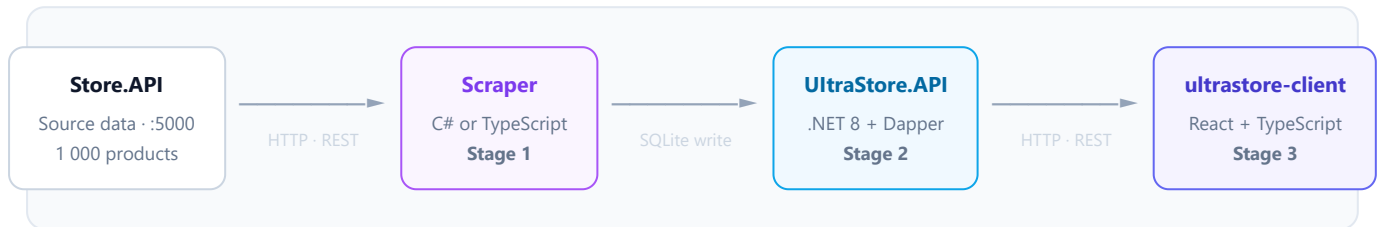
Work through each stage in order.
Your interviewer will reveal the next
stage when you are ready.

INTRODUCTION

Welcome to the UltraStore Assessment

You are joining the engineering team working on **UltraStore** — a product catalogue platform. The system exists and has been partially built. Your task is to implement the missing pieces, in the order they would naturally be built in production.

SYSTEM ARCHITECTURE



ALREADY IN PLACE

- ✓ Database schema (SQLite + PostgreSQL)
- ✓ API controller signatures and routing
- ✓ Repository layer with UPSERT method
- ✓ TypeScript type definitions
- ✓ React Query + React Router configured
- ✓ Axios client + Vite proxy to :5100
- ✓ Component structure and styles
- ✓ 1 000 seed products in Store.API

YOUR TASK

- 1 Build the data scraper — collect all products and persist them to the local database
- 2 Implement backend filtering, pagination, and input validation
- 3 Build the frontend — product listing, filtering, and detail page
- 4 Performance improvements and scalability discussion

RUNNING SERVICES

SERVICE	URL	NOTES
Store.API	http://localhost:5000	Source data — 1 000 seeded products. <i>Read-only, do not modify.</i>
UltraStore.API	http://localhost:5100	The backend you will implement. Reads from <code>scraper.db</code> .
React Dev Server	http://localhost:5173	Frontend. Vite proxies <code>/api/*</code> → <code>:5100</code> automatically.

Scraper & Data Collection

Build the data pipeline: collect all products from the Store.API and persist them to the local database. The scraper must be idempotent — running it twice must not create duplicate rows.

- Share this stage with the candidate at the start of the session.

FILE TO IMPLEMENT

C#:

`Store.Scraper/Services/ScraperService.cs`

TypeScript:

`store-scraper/src/scraperService.ts`

SOURCE API

GET

`http://localhost:5000/api/products`

Params: `page`, `pageSize`

Response includes `hasNextPage`

REPOSITORY METHOD

`UpsertProductAsync(product)`

Already implemented.

Handles INSERT + UPDATE.

Prevents duplicates automatically.

Requirements

- 1 Implement **ScrapeAsync()** (C#) or **scrape()** (TypeScript). The method stub already exists — fill in the logic.
- 2 **Paginate the source API**: request pages sequentially using `pageSize=100` until `hasNextPage` is false or the items array is empty.
- 3 For each product, assign a **fresh GUID** as the primary key (`Guid.NewGuid()` in C# / `crypto.randomUUID()` in TypeScript) and set `CreatedAt` and `UpdatedAt` to UTC now.
- 4 Persist each product by calling **UpsertProductAsync(product)**. Running the scraper a second time must not increase the product count.

Acceptance Criteria

CRITERION

- ☐ Running the scraper populates `scraper.db` with all 1 000 source products
- ☐ Running the scraper a second time does not increase the product count
- ☐ Each product has a valid GUID as its primary key
- ☐ `CreatedAt` and `UpdatedAt` are both set in UTC
- ☐ UltraStore.API returns the scraped products after the scraper runs

Note: If needed, your interviewer can provide a pre-populated `scraper.db` so UltraStore.API has data without completing this stage first. The scraper is independent of the other stages.

Backend Implementation

Implement server-side filtering and pagination in the repository, then add input validation in the controller that returns meaningful error responses before any database query is executed.

- Reveal after Stage 1 is complete (or when using a pre-populated database).

FILES TO EDIT

Data/ProductRepository.cs
Controllers/ProductsController.cs

METHOD SIGNATURE

```
GetProductsAsync(  
    search, category,  
    minPrice, maxPrice,  
    page, pageSize)
```

CRITICAL RULE

Use `DynamicParameters` for all filter values. Never concatenate user input into SQL strings.

Part A — Repository: Filtering & Pagination

- 1 Build a dynamic **WHERE clause** in `GetProductsAsync`. Add a condition only when a filter is non-null: **search** (case-insensitive LIKE on Name + Description), **category** (exact match), **minPrice** (Price $\geq$), **maxPrice** (Price $\leq$).
- 2 Use **DynamicParameters** for every filter value. All SQL inputs must be parameterized — no exceptions.
- 3 Run a **COUNT(*) query** with the same WHERE clause to populate the `TotalCount` field in the paginated response.
- 4 Add **LIMIT / OFFSET pagination**: `OFFSET = (page - 1) × pageSize`. Both values must be added as parameters.

Part B — Controller: Input Validation

Validate all query parameters *before* calling the repository. Return **HTTP 400** for each violation.

PARAMETER	RULE	ERROR MESSAGE
<code>page</code>	Must be ≥ 1	"page must be ≥ 1 ."
<code>pageSize</code>	Between 1 and 100 inclusive	"pageSize must be between 1 and 100."
<code>minPrice</code>	≥ 0 when provided	"minPrice must be ≥ 0 ."
<code>maxPrice</code>	≥ 0 when provided	"maxPrice must be ≥ 0 ."
<code>min</code> + <code>max</code>	<code>minPrice</code> $\leq$ <code>maxPrice</code> when both set	"minPrice must not exceed maxPrice."

★ BONUS

- + Return **ProblemDetails** (RFC 7807) instead of plain strings: `return Problem(detail: "...", statusCode: 400)`
- + Add a **database index** on `Name` in `InitializeAsync()` to improve search performance at scale

Frontend Implementation

Connect the React application to the API using React Query, implement filtering and search, then build a product detail page with client-side routing.

- Reveal after Stage 2. Work through Parts A, B, and C in sequence.

Part A — API Integration & Product Listing

FILES TO EDIT

`src/api/client.ts`
`src/api/products.ts`
`src/pages/ProductsPage.tsx`

ENDPOINTS

GET `/api/products`
 GET `/api/categories`
 Proxied via Vite to `:5100`

PRODUCTS RESPONSE

`{ items, totalCount,`
`page, pageSize,`
`hasNextPage }`

- 1 Configure the Axios instance in `src/api/client.ts` with **baseUrl**: `'/api'`. Vite proxies `/api` to `http://localhost:5100` — no CORS setup needed.
- 2 Implement **useProducts(filters)** and **useCategories()** in `products.ts` using `useQuery`. The `queryFn` must call the Axios client and return `response.data`.
- 3 Render the product grid in `ProductsPage.tsx` using `useProducts()`. Show loading and error states. Populate the category dropdown from `useCategories()`.

Part B — Filtering & Search

- 4 Pass all active filter values as **query parameters** to `GET /api/products`, and include them all in the **queryKey** — `queryKey: ['products', filters]` — so React Query re-fetches on any change.
- 5 Wire up the search input, category dropdown, and min/max price inputs. **Reset the page number to 1** whenever any filter changes.

Part C — Product Detail Page

FILES TO EDIT

`src/api/products.ts`
`src/pages/ProductDetailPage.tsx`

ENDPOINT & ROUTE

GET `/api/products/:id`
 Route `/products/:id` is already defined in `App.tsx`

FIELDS TO RENDER

Name · Image (with fallback)
 Category badge · Price
 Stock status · Description
 Created / Updated dates

- 6 Implement **useProduct(id)** using `useQuery` with `enabled: !!id` so the query only runs when an ID is present.
- 7 Build the **ProductDetailPage**: extract the product ID from the URL with `useParams()`, fetch the product, render all its fields, and handle loading and error states.
- 8 Navigate to the detail page when a product card is clicked (`/products/{id}`). Add a **Back button** that returns the user to the product list with their active filters intact.

★ BONUS

- + **Debounced search**: fire the query only after 350 ms of inactivity to avoid a request on every keystroke
- + **URL sync**: persist filter state in the URL with `useSearchParams` so reloading or sharing a link restores the same filters
- + **Cache pre-population**: use `queryClient.getQueryData` to `initialData` from the products list cache and avoid a loading flash on navigation
- + **Context API**: refactor filter state into `ProductContext` (scaffolded at `src/context/ProductContext.tsx`) to eliminate prop drilling

Improvements & Scalability

A small implementation task followed by an open architectural discussion. Demonstrate that you can reason about performance and trade-offs beyond the code you have written.

- Reveal after Stage 3. May be explored as a discussion if implementation time is limited.

IMPLEMENTATION

Add indexes in
`Data/ScraperRepository.cs`
→ `InitializeAsync()`

TARGET SCALE

10 million products
50 000 daily active users

APPROACH

Think out loud. Trade-offs matter more than naming every technology.

Part A — Implementation: Database Indexes

- 1 Add **CREATE INDEX** statement(s) to `InitializeAsync()` for the columns most commonly used in filters. Explain your choice.
- 2 Identify the **slowest query** in the current implementation at 1 million rows and explain why it would be slow without an index.

Part B — Architecture Discussion

Imagine UltraStore needs to scale to 10 million products with 50 000 daily users. Walk through how you would evolve each area below.

Search

When would you replace SQL LIKE with a full-text search engine? What are the trade-offs of PostgreSQL FTS vs Elasticsearch?

Pagination

When does offset pagination break down at scale? How would you migrate to cursor-based pagination without breaking existing clients?

Caching

What would you cache and where — HTTP Cache-Control, Redis, or a CDN? Which endpoints benefit most?

Scraper at Scale

The scraper currently runs once. At scale it needs to run continuously. Scheduled job, event queue, or change-data-capture?

Horizontal Scaling

Multiple API instances behind a load balancer — what changes? What about running the scraper on multiple nodes simultaneously?

Database

Should you keep SQLite at this scale? When would you move to PostgreSQL, introduce read replicas, or consider sharding?

-  **There are no right answers here.** A candidate who proposes one approach and clearly explains its trade-offs is more valuable than one who lists every technology without conviction.

Deliverables & Evaluation

What to submit and how your work is assessed

DELIVERABLES

- 1 **Source code** — the complete repository with your changes. Include all stages completed, even partially.
- 2 **Setup instructions** — how to run what you built (which services to start and in what order).
- 3 **Architectural notes** — 3–5 sentences on the key decisions you made and what you would do differently with more time.
- 4 **Known gaps** — anything intentionally skipped and why (time, scope, or deliberate trade-off).

HOW YOUR WORK IS EVALUATED

React & Frontend

- React Query queryKey structure
- Loading, error, and empty states handled
- Routing set up correctly
- TypeScript types used correctly

Backend & API Design

- Parameterized SQL (no injection risk)
- Validation before hitting the database
- Meaningful HTTP status codes
- Correct pagination and COUNT query

Code Quality

- Consistent naming conventions
- Clear single-responsibility functions
- No dead code or redundant logic
- Edge cases handled (null, empty, 0)

Engineering Thinking

- Articulates reasons for choices
- Acknowledges limitations of approach
- Considers scale implications
- Asks clarifying questions when needed